HDOC DAY-58

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k ≤ size. Pass the array to a function named kth_smallest() using its address. Use pointers to pass the array between the calling function and kth_smallest(). The function kth_smallest() must:

- Arrange the elements of arr[] in ascending order using bubble sort algorithm.
- Find the element that occupies the kth position in the sorted array.
- Display the kth smallest element.
- Consider duplicate elements as separate elements.
- Display -1 if k is invalid.

Example 1:

Array: 7 10 4 3 20 15
k: 3

Output:

k^{th} smallest element: 7

Example 2:

Array: 7 10 4 3 20 15
k: 4

Output:

k^{th} smallest element: 10

Example 3:

Array: 7 10 4 3 20 15
k: 7

Output: -1

Example 4:

Array: 4 2 2 7 5
k: 3

Output:

k^{th} smallest element: 4

[Explanation: The sorted array is 2 2 4 5 7, and duplicate elements are counted separately.]

Algorithm

1. Read size.
2. Declare a variable-length array arr[size].
3. Read all the elements of the array.
4. Read k.
5. Pass the address of the array to kth_smallest().
6. If k <= 0 or k >= size, display -1.
7. Otherwise, sort the array in ascending order using **Bubble Sort**.
8. Access the kth smallest element using *(arr + k - 1).
9. Display the kth smallest element.

• C Program:

```
• #include <stdio.h>
•
• void kth_smallest(int *arr, int size, int k)
• {
•    int i, j, temp;
•
•    // Check whether k is valid
•    if (k <= 0 || k >= size)
•    {
•       printf("-1");
•       return;
•    }
•
•    // Bubble sort using pointers
•    for (i = 0; i < size - 1; i++)
•    {
```

```
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }

    // kth smallest element
    printf("kth smallest element: %d", *(arr + k - 1));
}

int main()
{
    int size, k, i;

    scanf("%d", &size);

    int arr[size];

    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &k);

    kth_smallest(arr, size, k);

    return 0;
}
```

Test Case Table:

Test Case	Size	Array Elements	k	Sorted Array	Expected Output

1	6	7 10 4 3 20 15	3	3 4 7 10 15 20	Kth smallest element: 7
2	6	7 10 4 3 20 15	4	3 4 7 10 15 20	Kth smallest element: 10
3	6	7 10 4 3 20 15	7	-	-1
4	5	4 2 2 7 5	3	2 2 4 5 7	Kth smallest element: 4

Compilation and Execution:

The Following Program has been compiled and executed successfully.

Test Case 1:

```
┌──(kali㉿kali)-[~/Desktop
└─$ ./hdoc58
Enter array size: 6
Enter 6 array elements:
7 10 4 3 20 15
Enter k: 3
kth smallest element: 7
```

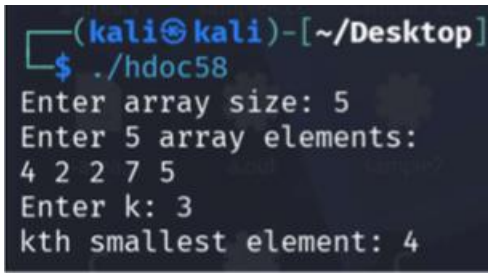
Test Case 2:

```
┌──(kali㉿kali)-[~/Desktop
└─$ ./hdoc58
Enter array size: 6
Enter 6 array elements:
7 10 4 3 20 15
Enter k: 4
kth smallest element: 10
```

Test Case 3:

```
┌──(kali㉿kali)-[~/Deskt
└─$ ./hdoc58
Enter array size: 6
Enter 6 array elements:
7 10 4 3 20 15
Enter k: 7
-1
```

Test Case 4:


(kali㉿kali)-[~/Desktop]
$ ./hdoc58
Enter array size: 5
Enter 5 array elements:
4 2 2 7 5
Enter k: 3
kth smallest element: 4